ADVANCED AI SOCIETY · PROOF-OF-CONTROL INITIATIVE
Working draft v0.1 · August 2026 · for co-author review prior to submission

Internal Consistency Is Not Evidence: Custody, Durability, and the Limits of a Tamper-Evident Log

A log that verifies against itself has told you nothing.

Jim Schwoebel[1,2]

[1] Quome [2] Advanced AI Society, `https://advancedaisociety.org`[1]

ABSTRACT

A tamper-evident log chains its records, roots them in a Merkle tree, and signs the root. Break any record and a link breaks. That is all true, and it is worth almost nothing against the party running the log, because that party does not break links—they recompute them. The rewritten log is not a damaged log. It is a well-formed log of different events, and every check it can perform on itself passes.

We say plainly what does the work instead. It is not cryptography; it is custody. A root that left the operator's hands before the rewrite is the only artefact that contradicts the rewrite, and a system with no such publication point is undefended no matter how carefully its proofs are implemented. We build the log anyway—chain, RFC 6962 tree, inclusion and consistency proofs verified exhaustively rather than sampled—and then make the tool print, on every root it emits, the sentence saying what it does not defend against.

Along the way we hit something we did not expect, and we think it is the more useful of our two findings. The requirement was fail-closed durability: never acknowledge a claim whose bytes are not on disk. The obvious way to test that is to make the store unavailable and check that everything is refused, and the obvious way to make it unavailable is to delete the file. That test passes while the system is broken. A process holding an open descriptor keeps writing after the path is unlinked, `fsync` returns success, and the log acknowledges claims backed by bytes nothing can ever read back. The fault injection injects no fault. We found it by luck—our assertion failed for the opposite reason—and we report the luck rather than dressing it up as method.

We also report that the schema we were implementing chains over two fields of a record rather than the record, which leaves the step index, the target resource, the policy bundle and every identity field outside the chain entirely.

[1] Prepared within the Proof-of-Control Initiative of the Advanced AI Society. Authorship is limited to contributors to this manuscript. Co-authors will be added only upon explicit consent and substantive contribution; the author list is deliberately short at this stage rather than inherited from a sibling paper.

1 The Problem

Somebody hands you a log. It records decisions an automated agent made: what it wanted to do, what the policy engine said, whether the action went through. Every record contains a hash of the record before it, so the records form a chain. The whole set is summarised by a Merkle root, and that root is signed. You check it. Every link holds. The signature is good.

What have you learned?

The honest answer is: that this log is well-formed. Not that it is true. Those are different, and the difference is the entire subject of this paper.

Here is why. Suppose the operator of the log wants record three to say something else. They are not going to alter record three and leave a broken link for you to find—that would be a strange kind of adversary, one who tampers but stops halfway. They alter record three, recompute the chain link for record four, and record five, and so on to the end. Then they sign the new root. Now the log has no broken link, because they fixed the links. It has no gap in the sequence, because they did not remove anything. The signature over the new root verifies, because they hold the key.

You are looking at a log that is, in every mechanical sense, perfect. And it is a log of events that did not happen.

I want to be careful about what is surprising here, because on one reading it is not surprising at all. Of course somebody with the signing key can sign things. That is what a key is. Everybody nods.

The trouble is that the nodding does not survive contact with a deployment. In practice you get a document that says the log is tamper-evident, and an auditor who verifies the log and reports that it checks out, and neither the document nor the auditor makes the distinction I just made. The verification step is real work, it produces a green tick, and the green tick means "well-formed," and everyone reads it as "true." The gap is not in anybody's understanding of cryptography. It is in what the tooling reports and what the report is taken to mean.

So we built the tool, and then we made the tool refuse to produce that green tick without a sentence next to it.

1.1 The one thing that does contradict a rewrite

There is exactly one artefact that catches this, and it is not clever.

If, before the rewrite, the operator published the root—handed it to a counterparty, wrote it into a transparency log, printed it and put it in a drawer—then that old root is a commitment they can no longer edit. A Merkle consistency proof between the old root and the new one either demonstrates that the new tree extends the old, or it does not exist, and against a rewritten history it does not exist. RFC 6962 defines exactly this proof for exactly this reason [4].

Notice what that costs. Not computation: the proof is logarithmic and the verification is a few dozen hashes. What it costs is a relationship. Somebody other than the operator has to have been holding something, before the fact, and be willing to say so afterwards. That is a custody arrangement and an organisational commitment, and no amount of engineering inside the log produces it.

This is the paper's central claim, and I want to state it in its blunt form before we dress it up:

> The security of a tamper-evident log against the party operating it is bounded above by the custody of its published roots, and is independent of the quality of its cryptography.

Independent. Not "mostly determined by." A log with a flawless RFC 6962 implementation and no publication point, and a log with a mediocre one and no publication point, are equally defenceless

against their operator. The cryptography is doing something—it defends against everyone who is not the operator, which is most attackers—but against the operator it contributes zero, and adding more of it continues to contribute zero.

1.2 Why this matters more for agents than it used to

Transparency logs have lived with this for a decade and solved it socially: Certificate Transparency has multiple independent logs, monitors that watch them, and auditors that gossip roots between clients [4, 5]. The design assumes the log operator is not trusted, and builds the ecosystem that makes the assumption safe.

The systems we are now being asked to log are not like that. An enterprise deploying an autonomous agent wants a record of what the agent did, for its own governance and for its regulator. The natural deployment is one log, run by the enterprise, holding records about the enterprise's own agents. Every party in that picture is the same party. The evidence that an agent did not exceed its authority is held, signed, and—if it comes to it—rewritten by the organisation whose interest is served by the answer.

I do not think this is usually malice. It is more often that nobody drew the diagram and noticed that the auditor and the audited are the same box.

2 What Others Have Built

The machinery here is old and good, and none of it is ours.

Merkle gave us the hash tree [6], which lets you commit to a set of things with one value and later prove membership without revealing the set. Haber and Stornetta gave us the linked timestamp [2], and stated the custody problem more clearly in 1991 than most systems state it now: their scheme distributes trust across clients precisely because a single timestamping service could collude with a client to backdate. That is our problem, thirty-five years earlier, and their answer was structural rather than cryptographic.

Certificate Transparency turned this into deployed infrastructure [4, 5]. The two proofs we implement are theirs: inclusion, that a record is in a tree with a given root; and consistency, that a tree of n leaves extends a tree of $m \leq n$ leaves. Crosby and Wallach worked through what tamper-evident logging costs in practice and where the naive data structures fall over [1].

On the durability side, the write-ahead log and the group commit are from database work of the seventies and eighties, with ARIES as the canonical write-up [7]. The idea we lean on—do not tell anybody the transaction committed until the bytes are on the disk—is the oldest idea in this paper.

What is newer, and more directly relevant to Section 4, is a line of work showing that the storage stack does not behave the way applications believe. Pillai and colleagues found that applications routinely depend on file-system behaviours that are not guaranteed, and that the resulting bugs survive testing because the tests share the application's assumptions [8]. Rebello and colleagues asked what happens when `fsync` itself fails and found that most applications handle it badly or not at all [9]. Our finding is a cousin of these: not that the storage stack lied, but that the *test* of the storage stack was a no-op for a reason nobody in the room had thought about.

Where we differ from all of this is emphasis rather than mechanism. The literature above mostly asks how to build the thing correctly. We are asking what a correctly built thing is worth in a deployment where the operator and the audited party coincide, and our answer is: less than the documentation says, and the shortfall is not fixable inside the tool.

3 Writing It Down Precisely

The symbols in this section will not make anything truer than the English already did. A reader who skips to Section 4 loses only precision. But it is worth being exact about which checks depend on what, because the whole argument is that two of them look alike and are not.

Let a log be a sequence of records $R = (r_0, \ldots, r_{n-1})$. Write $d(r)$ for the digest of a record's canonical form—RFC 8785 [10], computed by a shared implementation rather than a second one, for reasons the sibling paper works through—and define the chain by $h_0 = 0^{256}$ and $h_{i+1} = H(h_i \| d(r_i))$. Let $\mathrm{MTH}(R)$ be the RFC 6962 Merkle tree head over the leaf hashes of R.

Definition 1 (Internal consistency)**.** *R is internally consistent if each r_i declares chain head h_i, and the step indices of R are exactly $0, \ldots, n-1$.*

Definition 2 (Extension)**.** *R' extends R if R is a prefix of R'.*

Now the two propositions that matter, and the gap between them.

Proposition 1 (Internal consistency is cheap to forge)**.** *For any sequence of claims C and any modification of any claim in C, there is an internally consistent log recording the modified sequence, computable in time linear in $|C|$ by any party able to sign a root.*

The proof is the attack: recompute the chain from the modified record forward, rebuild the tree, sign. Nothing is hard. We ship it as a runnable example rather than as an argument, because a reader can then watch every internal check return true.

Proposition 2 (Consistency against a prior root is not)**.** *Let $\rho = \mathrm{MTH}(R)$ be published at size m. If R' of size $n \geq m$ does not extend R, then no consistency proof between (m, ρ) and $(n, \mathrm{MTH}(R'))$ verifies, under collision resistance of H.*

This is RFC 6962's property, not a new one [4]. We restate it only to make the asymmetry visible: Proposition 1 needs the signing key and nothing else, and Proposition 2 needs an artefact that escaped the operator at a particular time. The first is a computation. The second is a fact about the world.

> **The custody condition.** A log detects operator rewriting at size m if and only if some party outside the operator's control holds $\mathrm{MTH}(R_{\leq m})$ and will produce it on demand.

I have written that as a biconditional and I want to flag that the "only if" direction is the interesting one and the one we can actually argue: with no external holder, the operator's view of history is the only view, and any internally consistent log is consistent with it. The "if" direction assumes the external party is honest and available, which is an organisational assumption doing a lot of quiet work. We have not formalised what happens when the external holder is partly trusted, or when there are several disagreeing ones. That is where the Certificate Transparency ecosystem put a decade of effort [5], and we have not repeated it.

4 The Test That Proves Nothing

Now the part we did not see coming.

Separately from all the Merkle machinery, the log has a durability requirement, and it is the requirement the whole component exists for: never acknowledge a claim whose bytes are not durably written. If an enforcement point is waiting on this acknowledgement before letting an action proceed,

then acknowledging early means an action happens with no evidence that it happened. Fail-closed, no exceptions, no flag to turn it off.

The implementation is standard. Serialise the batch, one `write`, one `fsync`, and only then construct the acknowledgement. We made the acknowledgement type constructible in exactly one place in the codebase, after the sync returns, so that "acknowledge before durable" is not an expressible program rather than a discouraged one.

Then we wrote the test. The requirement says: make the evidence store unavailable, and observe that actions do not proceed. So: create the log, take the store away, submit some claims, assert that every one of them is refused.

How do you take the store away? You delete the file. That is the obvious move, and it is what we did—unlink the segment and put a directory in its place, so that any attempt to re-open it fails.

The test failed. Every claim was acknowledged.

Here is what is going on, and it is embarrassing in the specific way that good findings often are. On a POSIX system, a file descriptor refers to an inode, not to a path. Unlinking the path removes the directory entry. It does not touch the descriptor. Our committer had opened the segment at start-up and was still holding that descriptor, so:

```
write to unlinked fd: Ok(())
reopen at same path: Err(Os { code: 21, kind: IsADirectory })
```

The write succeeds. The `fsync` succeeds. The bytes really are on the disk, in an inode with no name, which the kernel will free the moment the last descriptor closes. The durability requirement is satisfied in the narrowest possible reading—the bytes were synced—and violated in every sense anybody cares about, because the evidence cannot be read back by anything, ever.

So the log acknowledges the claim. The enforcement point lets the action through. The action happens, and there is no record of it, and no error was raised anywhere in the system.

4.1 Why we want to make a fuss about this

The bug is not that unlinked descriptors keep working. That is specified behaviour—`unlink` removes the directory entry and defers destroying the file until the last descriptor is closed [3]—and any systems programmer asked directly will tell you. We confirmed it on macOS with the standalone probe above and have not run it elsewhere. The finding is about the test.

We wrote a fault-injection test. It injected no fault. The store was never unavailable; only its name was. And the assertion the test existed to check was therefore never exercised at all. Had our implementation been slightly different—had we, say, cached the acknowledgement path in a way that happened to refuse—the test would have passed, and we would have shipped a green fail-closed test over a durability path that had never once been made to fail.

That is the shape that worries me. A test that passes for the wrong reason is worse than a missing test, because the missing test is visible in a coverage report and the wrong one is not. And this particular wrong reason is attractive: "delete the file" is the first thing anybody tries.

4.2 How we actually found it, which was luck

I want to be accurate about the process, because the tidy version would be dishonest.

We did not find this by reasoning about POSIX. We found it because the assertion failed—and it failed in the direction that says "you acknowledged something you should have refused," which is alarming enough that we went looking at the mechanism instead of the implementation. If the test had passed, we would have ticked it off and moved on. The mechanism was only inspected because the outcome was already wrong.

So the honest summary is: a bug in our fault injection was caught by an unrelated bug in the same test, and if either had been absent we would have shipped. We wrote a standalone probe afterwards to confirm the POSIX behaviour directly rather than infer it from the failure, and that probe is the listing above.

4.3 The fix, and what it does not fix

The repair is not in the test. A committer that cannot re-open its store now refuses everything, because a descriptor that has outlived its path is exactly the condition under which acknowledging is unsafe. That is production behaviour, not a test affordance: the same reasoning applies if an operator moves a log directory under a running process.

What we have not done is establish that our durability path is correct under real crash conditions. Refusing when the store vanishes is one narrow case. The literature on crash-consistency testing [8, 11] exists because doing this properly means enumerating reorderings a real storage stack permits, and we have not done that. We tested that a torn trailing record is truncated and that a corrupted complete record is refused, by writing the damage ourselves. That is a simulation of a crash, not a crash.

5 How We Checked

The proofs are the part of this system where a subtle error is both easy to make and hard to notice, so they get the disproportionate share of the testing, and the testing is exhaustive rather than sampled.

For every tree size from 1 to 33, and for every leaf in the tree, we generate an inclusion proof and verify it. For every tree size from 1 to 33, and every prefix length from 1 to that size, we generate a consistency proof and verify it. Separately, the incremental root—maintained as a right fringe as leaves arrive—is compared against a recursive reference implementation written straight from the RFC, at every size.

The bound of 33 is chosen to get past the powers of two. RFC 6962 splits every interior node at the largest power of two strictly below the node's size, and an implementation that gets this subtly wrong is usually still correct at 2, 4, 8, 16 and 32. A test suite that checks those sizes finds nothing. Sizes 17 and 33 are where the fringe logic has to do something interesting, which is why the range ends just past one.

The discipline we set ourselves, and kept, was: if a proof test fails, fix the verifier against the RFC, never relax the test. This sounds obvious written down. It is less obvious at 2am when the exhaustive test fails at size 23 and the tempting move is to check a narrower property. We record it here because the sibling project's history contains exactly that failure: a verifier that compared record counts instead of roots, and consequently accepted a rewritten history, which is the attack in Section 1 passing a test suite.

We also assert the negative controls, which matter more than they look. It is easy to write a verifier that rejects a rewritten log; the empty verifier that rejects everything does that. So there is a test that an *honest* extension of the same prior root is accepted, and a test asserting that a rewritten log is accepted when no prior root is supplied—because that acceptance is the honest limitation, and if it ever changed silently, the tool would be claiming a defence it does not have.

6 The Chain That Does Not Cover The Record

One finding about the specification we were implementing, which is smaller than Section 4 but easier to act on.

The evidence schema defines the chain head as

$$h_{i+1} = H(h_i \parallel \texttt{canonical_snapshot_hash}_i \parallel \texttt{verdict}_i)$$

that is, over two fields of the record plus the previous head. Not over the record.

So the chain protects those two fields. Everything else in the record is outside it: the step index, the target resource, the policy bundle hash, the agent identifier, the initiating user, the timestamp. An operator—or anyone who can write to the store—can change which resource an action touched, or which user initiated it, or which policy bundle was in force, and every chain link still verifies, because none of those values is an input to any link.

That is very likely an oversight rather than a decision; a snapshot hash and a verdict are the two fields that feel like "the decision," and it is easy to write the definition around them. But the consequence is that a log which passes its own chain check has not established the integrity of most of its content.

We chain over the whole record digest instead: $h_{i+1} = H(h_i \| d(r_i))$. It is strictly stronger, and it is free—$d(r_i)$ is already computed to form the Merkle leaf, so this costs one hash we were performing anyway. We report the schema definition upstream as a finding rather than implement it. An implementation that honoured it would be conformant and weaker, and we would rather be non-conformant in the direction of covering more of the record.

7 What We Still Don't Know

The single largest gap is the one this paper is about, and we have not closed it: we have no publication point. Everything here reduces to the custody condition, and the custody condition is an arrangement between organisations that a Rust crate cannot create. What the crate can do—and does—is refuse to let a root be printed without the sentence saying so, so that nobody reads a green tick as more than it is. That is a documentation defence against a structural problem, and I would not want to oversell it.

We have no performance numbers. None. The log batches claims into a single sync and we have not measured what that costs, so we do not know whether the durability requirement is compatible with the action rates people want from agents, and we are not going to guess. Anyone who needs that number should treat this paper as silent on it rather than as reassuring.

We have not measured the linkability cost of the tree topology. A single global tree reveals nothing about who acted, because membership distinguishes nobody. A tree per agent parallelises well and its key *is* an agent identifier, so every action in it is linked to every other by construction. Those two facts point in opposite directions and we have implemented all three arrangements without measuring either axis.

The signer is a software key, and we should be blunt that this means the whole system currently has no defence against its operator at all—the custody argument above is not mitigated by the signer, it is the reason the signer's weakness does not much matter yet. A key in a trusted execution environment would change who can produce a signature, not who can publish a root, so it narrows the attack without closing it.

Finally, on the finding in Section 4: we do not know how common it is. We found one instance, in our own code, by accident. The reason we think it generalises is that the reasoning that produced

our broken test—"to make a resource unavailable, delete it"—is not idiosyncratic, and the failure is silent. But we have not surveyed other durability test suites for the same pattern, and until somebody does, "we think this is common" is a hypothesis and should be read as one.

Availability

The implementation, the runnable demonstration of the rewrite attack, and the source of this paper are at `https://github.com/Task-force-for-AI-agents-in-Healthcare/ephemeris`, under Apache-2.0.

References

[1] Scott A. Crosby and Dan S. Wallach. Efficient data structures for tamper-evident logging. In *Proceedings of the 18th USENIX Security Symposium*, 2009.

[2] Stuart Haber and W. Scott Stornetta. How to time-stamp a digital document. *Journal of Cryptology*, 3(2):99–111, 1991.

[3] IEEE and The Open Group. *IEEE Std 1003.1-2017, Standard for Information Technology — Portable Operating System Interface (POSIX)*, 2018.

[4] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, IETF, 2013.

[5] Ben Laurie, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, IETF, 2021.

[6] Ralph C. Merkle. Protocols for public key cryptosystems. In *Proceedings of the 1980 IEEE Symposium on Security and Privacy*, pages 122–134, 1980.

[7] C. Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems*, 17(1):94–162, 1992.

[8] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. All file systems are not created equal: On the complexity of crafting crash-consistent applications. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 433–448, 2014.

[9] Anthony Rebello, Yuvraj Patel, Ramnatthan Alagappan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Can applications recover from fsync failures? In *Proceedings of the 2020 USENIX Annual Technical Conference*, 2020.

[10] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON canonicalization scheme (JCS). RFC 8785, IETF, 2020.

[11] Mai Zheng, Joseph Tucek, Dachuan Huang, Feng Qin, Mark Lillibridge, Elizabeth S. Yang, Bill W. Zhao, and Shashank Singh. Torturing databases for fun and profit. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 449–464, 2014.